Advanced Computer Networks — Final Exam Study Note

Scope: The final is **comprehensive** — it covers the midterm topics **plus** the post-midterm material up to the link layer. So this note has:

- **Part 0 — Midterm topics (recap):** Ch. 1 (delays, throughput, statistical multiplexing), Ch. 2 (HTTP, DNS, web caching), and the Network Layer (IP addressing, subnetting, VLSM, NAT).
- **Part A — Chapter 3:** the Transport Layer (UDP, TCP, reliable data transfer, flow & congestion control).
- **Part B — Chapter 6:** the Link Layer & LANs (error detection, multiple access, MAC/ARP, Ethernet, switches, VLANs).
- **Part C — Solved assigned questions:** Ch. 3 R3, R4, R14, R15, R18 and P1, P27, P28, P40; Ch. 6 R1, R2, R3, R4, R10, R11.

All from Kurose & Ross *Top-Down Approach* (9th ed.) plus your course slides. Formulas you must memorize are in `code boxes`; common exam traps are flagged in callouts.

Interactive companion: open `browsing_the_internet.html` (in this folder) — a live "Browsing the Internet" simulator. Drag sliders for link rate, distance, RTT, packet size, HTTP mode, and cache hit rate, and watch the DNS → TCP handshake → HTTP request/response timeline animate with the exact delay math used in this note.

PART 0 — MIDTERM TOPICS (COMPREHENSIVE RECAP)

The final is cumulative, so this part condenses the midterm-era material. It is **problem-oriented** — practice the math, don't just memorize definitions.

0.1 Chapter 1 — Internet basics, delay, throughput

Three pieces of the Internet: end systems / hosts (run apps at the edge), packet switches (routers & switches in the core), and communication links (fiber, copper, radio).

Protocol = rules defining the **format**, **order** of messages, and **actions** on send/receive. Standards are **RFCs** written by the **IETF**.

Packet vs circuit switching:

Packet switching	Circuit switching
Data split into packets	Dedicated path reserved end-to-end
Resources **shared** (statistical multiplexing)	Resources **reserved** (idle time wasted)
No setup; variable delay; possible loss	Needs setup; predictable delay; rarely lost
Used in the Internet	Old telephone network

Four sources of packet (nodal) delay

```
d_nodal = d_proc + d_queue + d_trans + d_prop

d_proc  : processing (check errors, choose output link)  ~ constant, µs
d_queue : waiting in the output buffer                    VARIABLE (depends on congestion)
d_trans : L / R   push all bits onto the link             constant for fixed L,R
d_prop  : d / s   bits travel down the link               constant for fixed link
```

- **d_trans = L/R** depends on packet length L and link rate R.

- **d_prop = d/s** depends on distance d and signal speed s (≈ 2–3 ×10■ m/s). It does **not** depend on L or R.

> **Exam trap:** transmission vs propagation. Pushing a long train onto the track (transmission) is different from the train riding down the track (propagation). Only `d_queue` is variable.

Store-and-forward (one switch between hosts, ignore prop/queue/proc): the switch must fully receive a packet before resending it, so the transmissions are in series:

```
d_end-to-end = L/R1 + L/R2
```

Queueing delay for a packet with n ahead of it (x bits of the in-progress packet already sent):

```
d_queue = ((n+1)·L − x) / R
```

Throughput and the bottleneck link

```
throughput = min(R1, R2, …, Rn)      time to move a file F = F / throughput
```

The **slowest link** sets the pace; a path is only as fast as its narrowest pipe.

Statistical multiplexing (worked example)

Q: A **1 Gb/s** link is shared by N users. Each needs **100 Mb/s** when active and is active **10%** of the time. (a) How many users under circuit switching? (b) Under packet switching, is 35 users safe?

```
(a) Circuit switching reserves r per user forever:
    N_max = C / r = 1000 Mb/s / 100 Mb/s = 10 users

(b) Packet switching: link can carry at most n* = C/r = 10 active users at once.
    Active users X ~ Binomial(N=35, p=0.10),  E[X] = N·p = 3.5
    P(X > 10) = Σ_{k=11..35} C(35,k)·0.1^k·0.9^(35−k) < 0.0004
    → 35 users share safely; congestion chance < 0.04%.
```

> **Exam trick:** "users active X% of the time" → expected active = N·p, then check P(active > capacity) with the binomial.

0.2 Chapter 2 — Application layer (HTTP, DNS, caching)

Socket = IP address : port number. The IP identifies the host (building); the 16-bit port identifies the process inside it (apartment).

Transport choice — TCP vs UDP:

	TCP	UDP
Connection	3-way handshake	connectionless
Reliability	reliable, in-order, retransmits	best-effort, may lose/reorder
Flow & congestion control	yes	no
Header	20+ bytes	8 bytes
Uses	HTTP, HTTPS, SMTP, FTP, SSH	DNS, VoIP, live video, games

HTTP

HTTP runs over TCP and is **stateless** (server keeps no memory of past requests; cookies add state on top).

- **Non-persistent HTTP:** one object per TCP connection, then close → each object costs a new handshake. Time per object ≈ **2·RTT + transmit time** (1 RTT to set up TCP, 1 RTT for request/response).
- **Persistent HTTP (default in 1.1):** many objects reuse one TCP connection → far fewer setups, lower delay.

Status codes: 200 OK, 301 Moved Permanently, 304 Not Modified (conditional GET), 400 Bad Request, 404 Not Found, 505 Version Not Supported.

Cookies: Set-Cookie header (response) → Cookie header (later requests) → browser cookie file → server-side DB. Used for login/session, carts, recommendations.

Web caching (proxy) — access-link math (classic exam problem)

A cache serves hits locally (fast) and forwards misses to the origin (and stores a copy). It cuts the traffic on the expensive access link.

Q: Access link **1.54 Mbps**; average object **100,000 bits**; request rate **15 req/s**; RTT to origin **2 s**.

```
Offered traffic  T = λ · L = 15 × 100,000 = 1.5 Mb/s
Utilisation      U = T / C = 1.5 / 1.54 = 0.97   → as U→1, queue delay → ∞  (unusable)

Add a cache, hit rate h = 0.4 → only misses use the link:
T' = (1–h)·λ·L = 0.6 × 1.5 Mb/s = 0.9 Mb/s
U' = 0.9 / 1.54 = 0.58   (small queue)
Avg delay = (1–h)·d_miss + h·d_hit ≈ 0.6×2.01 + 0.4×0.001 ≈ 1.2 s
```

Exam trap: utilisation `U = offered traffic / capacity`. Once U approaches 1, queueing delay blows up. A cache reduces offered traffic by the hit fraction.

DNS — the Internet's phone book

Translates names (`www.google.com`) → IP addresses. It is **distributed & hierarchical** (root → TLD → authoritative) so it avoids a single point of failure, traffic overload, distant lookups, and unmaintainability.

Resolution: browser → local resolver → root ("ask .com") → TLD ("ask example.com's authoritative") → authoritative (returns the A record). Caching (with TTLs) short-circuits later lookups.

Record types: A (name→IPv4), AAAA (name→IPv6), CNAME (alias), MX (mail server), NS (authoritative name server).

0.3 Network Layer — IP addressing, subnetting, VLSM, NAT

IPv4 = 32 bits = four octets (0–255). Split into a **network portion** (left) and **host portion** (right); the **subnet mask** / CIDR `/m` marks where the split is (m network bits).

Classful ranges (identify by first octet): A = 0–127 (/8), B = 128–191 (/16), C = 192–223 (/24), D = 224–239 (multicast), E = 240–255 (reserved). Private ranges: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`. Special: `127.0.0.1` loopback, `255.255.255.255` broadcast, `0.0.0.0` default/unknown.

The block-size method (no long binary needed)

```
host bits  n = 32 – m
usable hosts = 2^n – 2          (minus network + broadcast)
block size = 256 – (last non-255 octet of the mask)
subnets start at multiples of the block size in that octet
network  = block start;   broadcast = next block start – 1
```

Worked — `172.16.35.123/20`**:** mask `255.255.240.0`, n = 12 host bits, block = 256–240 = 16 in the 3rd octet. 35 falls in [32,47]:

```
Network   = 172.16.32.0
Broadcast = 172.16.47.255
Host range= 172.16.32.1 – 172.16.47.254
Hosts     = 2^12 – 2 = 4094
```

Common masks: /25 = .128 (126 hosts), /26 = .192 (62), /27 = .224 (30), /28 = .240 (14), /29 = .248 (6), /30 = .252 (2).

VLSM — Variable Length Subnet Masks

Give each subnet its own mask, just big enough. **Recipe:** sort requirements **largest** → **smallest**; for each pick the smallest n with `2^n – 2 ≥ hosts`; assign the block; start the next subnet right after the previous broadcast.

Worked — split `192.168.10.0/24` **for LAN1=100, LAN2=50, LAN3=20:**

LAN	Need	2■–2	Prefix	Network	Broadcast	Host range
LAN1	100	126 (n=7)	/25	192.168.10.0	192.168.10.127	.1 – .126
LAN2	50	62 (n=6)	/26	192.168.10.128	192.168.10.191	.129 – .190
LAN3	20	30 (n=5)	/27	192.168.10.192	192.168.10.223	.193 – .222

Exam trap: always sort biggest-first, or a small subnet will land on a boundary the big one needs. Remember 2■ – 2 (reserve network + broadcast).

NAT — Network Address Translation

Lets a whole private network share **one** public IPv4 address. The router rewrites **(source IP, source port)** on the way out and reverses it on the way back, tracking each mapping in a NAT translation table.

Inside (private)	↔	Outside (public)
192.168.1.5:5000	↔	203.0.113.5:30001 → google:443
192.168.1.6:5000	↔	203.0.113.5:30002 → youtube:443

Pros: conserves IPv4, hides internal topology, basic security. **Cons:** breaks end-to-end addressing, complicates P2P/gaming, extra router work.

PART A — CHAPTER 3: THE TRANSPORT LAYER

3.1 What the transport layer does

The transport layer gives **logical communication between application processes** running on different hosts. The network layer (IP) only gives logical communication between **hosts**; the transport layer extends host-to-host delivery to **process-to-process** delivery.

- **Sender side:** takes an application message, breaks it into **segments**, adds a transport header, hands each segment to IP.
- **Receiver side:** reassembles segments into the message and delivers it to the correct socket (application).

Two Internet transport protocols:

	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless (no handshake)
Reliability	Reliable, in-order byte stream	Unreliable, may lose / reorder
Flow control	Yes	No
Congestion control	Yes	No
Extras	Full-duplex, point-to-point	"Bare bones" best-effort
Neither provides	delay guarantees, bandwidth guarantees	(same)

Analogy (kids-in-two-houses): IP delivers a letter to the right *house* (host). The transport layer is the *sibling* who hands the letter to the right *person* (process) inside the house.

3.2 Multiplexing and demultiplexing

- **Multiplexing (sender):** gather data from many sockets, add header info, pass segments down.
- **Demultiplexing (receiver):** use header fields to deliver a segment to the correct socket.

How demux works: each host receives IP datagrams; each datagram carries source/dest **IP address**; each segment carries source/dest **port number**.

- **UDP demux uses a 2-tuple:** `(dest IP, dest port)`. Two segments with the **same dest port** but different source IPs/ports go to the **same** UDP socket.
- **TCP demux uses a 4-tuple:** `(source IP, source port, dest IP, dest port)`. A web server has a **different connection socket per client**, even though all use dest port 80.

> **Exam trap:** For UDP, the source port is only used as a *return address*; it does NOT change which socket receives the data. For TCP, all four fields matter.

3.3 UDP — User Datagram Protocol (RFC 768)

"No frills" best-effort transport. Segments may be lost or delivered out of order. **Connectionless:** no handshake, each segment handled independently.

Why choose UDP?

- No connection setup → no extra RTT of delay.
- No connection state at sender/receiver → supports more active clients.
- Small header (8 bytes).
- Fine-grained application control over what/when is sent (no congestion throttling).

Used by: DNS, SNMP, streaming multimedia (loss-tolerant, rate-sensitive), HTTP/3 (QUIC adds reliability on top of UDP at the app layer).

UDP header = 8 bytes: source port, dest port, length, checksum.

UDP checksum — detects flipped bits:

1. Treat segment contents (plus some header fields) as a sequence of 16-bit integers.
2. Add them with **one's-complement** addition; any carry-out of the top bit is **wrapped around** and added back.
3. Take the one's complement of the sum → that is the checksum.
4. Receiver adds everything **including** the checksum; if the result is all 1s → no error detected; otherwise error.

> Checksum is *weak* protection: it can miss some multi-bit errors (e.g., two compensating flips).

3.4 Principles of reliable data transfer (rdt)

We build a reliable channel on top of an unreliable one, step by step:

Protocol	Channel assumption	Mechanism added
rdt1.0	Perfectly reliable	Nothing needed
rdt2.0	Bit errors	Checksum + **ACK/NAK** (retransmit on NAK)
rdt2.1	Errors incl. corrupt ACK/NAK	**Sequence numbers** (0/1) to catch duplicates
rdt2.2	Same, NAK-free	ACK carries seq # of last good pkt; **duplicate ACK = NAK**
rdt3.0	Errors **and loss**	**Countdown timer** → retransmit on timeout

Key ideas you must be able to state:

- **Sequence numbers** let the receiver tell a *new* packet from a *retransmission* (duplicate).
- **Timers** handle *loss*: if no ACK arrives within the timeout, assume the packet (or its ACK) was lost and retransmit.
- rdt3.0 is a **stop-and-wait** protocol (alternating-bit): only 1 packet in flight at a time.

Stop-and-wait is slow. Utilization:

```
            L/R
U_sender = ---------
          RTT + L/R
```

Example: 1 Gbps link, 15 ms one-way prop (RTT = 30 ms), L = 8000 bits. `L/R = 8 µs`, so `U = 0.008 / (30.008) ≈ 0.00027` = **0.027%**. Terrible — the protocol, not the link, limits throughput.

Pipelining: Go-Back-N vs Selective Repeat

Allow multiple unACKed ("in-flight") packets. Sequence-number range must grow; buffering needed.

Go-Back-N (GBN):

- Sender window of up to N unACKed packets, one timer (for the oldest unACKed).
- **Cumulative ACK:** ACK(n) acknowledges *all* packets up to and including n.
- On timeout: **retransmit packet n and every higher packet in the window**.
- Receiver is simple: accepts only **in-order** packets, discards out-of-order ones, re-ACKs the highest in-order seq # received. No receiver buffering.

Selective Repeat (SR):

- Sender keeps a **timer per unACKed packet**; on timeout retransmits **only that one** packet.
- Receiver **individually ACKs** every correctly received packet and **buffers** out-of-order packets for later in-order delivery.
- **Window constraint:** `window size ≤ (sequence-number space) / 2`, otherwise the receiver cannot tell a new packet from a retransmission.

> **Exam trap:** GBN uses a single cumulative ACK and one timer; SR uses individual ACKs, per-packet timers, and a receiver buffer.

3.5 TCP — reliable, connection-oriented transport

RFCs 793, 1122, 2018, 5681, 7323. Properties: point-to-point, reliable in-order **byte stream** (no message boundaries), full-duplex, connection-oriented, flow-controlled, pipelined, cumulative ACKs. **MSS** = maximum segment size (payload).

Segment header highlights: source & dest ports; **sequence number**; **acknowledgment number**; header length; flags (SYN, ACK, FIN, RST, PSH, URG); **receive window (rwnd)**; checksum.

Sequence & ACK numbers (byte-oriented):

- Seq # = byte-stream number of the **first byte** in the segment's data.
- ACK # = seq # of the **next byte expected** from the other side (cumulative).

RTT estimation and timeout:

```
EstimatedRTT = (1 - α)·EstimatedRTT + α·SampleRTT     (α = 0.125)
DevRTT       = (1 - β)·DevRTT + β·|SampleRTT - EstimatedRTT|   (β = 0.25)
TimeoutInterval = EstimatedRTT + 4·DevRTT
```

- SampleRTT = time from sending a segment until its ACK (ignore retransmitted segments).
- EWMA smooths noisy samples; the 4·DevRTT "safety margin" grows when RTT is jittery.

Fast retransmit: on receiving **3 duplicate ACKs** for the same data, the sender retransmits the missing segment *before* the timer expires.

Retransmission scenarios to know: lost ACK, premature timeout (sender resends but a cumulative ACK later covers everything), cumulative ACK covering an earlier lost ACK.

TCP flow control

Keeps the **sender from overflowing the receiver's buffer** (a receiver-speed issue, *not* a network issue).

- Receiver advertises free buffer space in the **rwnd** field of every ACK.
- Sender limits in-flight (unACKed) data to ≤ rwnd → receive buffer never overflows.

> **Exam trap:** Flow control ≠ congestion control. Flow control protects the *receiver*; congestion control protects the *network*.

TCP connection management

3-way handshake (open):

1. Client → Server: `SYN`, seq = x (no data).
2. Server → Client: `SYNACK`, seq = y, ACK = x+1.
3. Client → Server: `ACK`, seq = x+1, ACK = y+1 (may carry data). Both now **ESTABLISHED**.

A 2-way handshake is *not* enough: variable delays, retransmissions, and reordering can create half-open or duplicate connections.

Closing: each side sends `FIN`, the other replies `ACK`; a received FIN can be combined with the sender's own FIN.

3.6 Principles of congestion control

Congestion = "too many sources sending too much data too fast for the network to handle." Symptoms: long queueing delays and packet loss (router buffer overflow).

Costs of congestion (from the scenarios):

1. Large queueing delays as arrival rate → link capacity.
2. Retransmissions needed for lost packets (extra work per delivered byte).
3. **Unneeded** retransmissions from premature timeouts → link carries multiple copies.
4. When a packet is dropped after several hops, **all upstream capacity used on it is wasted.**

Two approaches:

- **End-to-end** (TCP's approach): no help from the network; congestion **inferred** from loss/delay.
- **Network-assisted:** routers signal congestion (e.g., **ECN** bits in the IP header, ATM, DECbit).

3.7 TCP congestion control

TCP adjusts a **congestion window (cwnd)**; the sender limits in-flight data to `min(cwnd, rwnd)`.

AIMD — Additive Increase, Multiplicative Decrease (the "sawtooth"):

- **Additive increase:** +1 MSS per RTT while no loss (probing for bandwidth).
- **Multiplicative decrease:** on loss, cut the rate.
- Loss by **triple duplicate ACK** → cwnd cut to ½ (TCP Reno).
- Loss by **timeout** → cwnd cut to **1 MSS** (more drastic, TCP Tahoe behavior).

Slow start: at connection start cwnd = 1 MSS and **doubles every RTT** (exponential) — cwnd increases by 1 MSS for each ACK — until the first loss or until cwnd reaches **ssthresh**.

Slow start → congestion avoidance: when cwnd reaches ssthresh, switch from exponential to **linear** (additive) growth. On a loss event, `ssthresh = cwnd/2`.

TCP throughput (rough): if W is the window (bytes) at which loss occurs, average window ≈ ¾W, so

```
avg throughput ≈ (3/4)·W / RTT
```

TCP fairness: K TCP flows sharing a bottleneck of rate R each tend toward **R/K**. AIMD drives competing flows toward the equal-share point. Caveats: UDP flows don't back off; an app opening **many parallel TCP connections** gets a larger share.

Newer congestion control: TCP CUBIC (window grows as a cubic function of time since last loss, approaching the previous max W_max cautiously); BBR (models bottleneck bandwidth and min-RTT to keep the pipe "just full, not fuller"); QUIC (HTTP/3) moves reliability + congestion control into the app layer on top of UDP.

PART B — CHAPTER 6: THE LINK LAYER AND LANs

6.1 Introduction and services

- **Nodes:** hosts and routers. **Links:** channels connecting adjacent nodes (wired, wireless, LANs).
- Layer-2 packet = **frame**, which encapsulates a datagram.
- A datagram may travel over **different link protocols** on different links (e.g., WiFi then Ethernet); each provides different services.

> **Transportation analogy:** trip Princeton→Lausanne. Tourist = datagram; each transport leg (limo/plane/train) = a link; the mode = link-layer protocol; travel agent = routing algorithm.

Link-layer services:

- **Framing & link access:** encapsulate datagram, add header/trailer; use MAC addresses to identify source/dest on the link; coordinate access if the medium is shared.
- **Reliable delivery** between adjacent nodes (common on high-error wireless links, rarely used on low-error wired links).
- **Error detection** (drop/re-request bad frames) and **error correction** (fix bits without retransmission).
- **Flow control**, and **half/full-duplex** operation.

Where implemented: in the **network interface card (NIC)** / on-chip "adapter" — a mix of hardware, software, and firmware attached to the host bus.

6.2 Error detection and correction

Sender adds **EDC** (error-detection-and-correction) bits to data **D**. Detection is **not 100% reliable** — a larger EDC field detects/corrects more.

- **Parity:** single-bit parity detects any odd number of bit errors. **Two-dimensional parity** can **detect and correct** a single-bit error (it pinpoints the row and column).
- **Internet checksum:** (see §3.3) used at transport layer; cheap in software but weak.
- **Cyclic Redundancy Check (CRC):**
- Given data D and an agreed (r+1)-bit generator G, the sender finds r CRC bits R so that `<D,R>` is exactly divisible by G modulo 2 (XOR arithmetic).
- Compute R as the remainder of `D·2^r` divided by G (mod 2).
- Receiver divides the received `<D,R>` by G; **nonzero remainder** ⇒ **error**.
- **Detects all burst errors of length** ≤ **r.** Used by Ethernet and 802.11 WiFi.

6.3 Multiple access protocols

Two link types: **point-to-point** (one sender, one receiver) and **broadcast** (shared medium — old Ethernet, cable upstream, 802.11 WiFi, satellite).

Problem: a shared broadcast channel — two simultaneous transmissions **collide**. A multiple-access protocol is a **distributed** algorithm deciding when each node may transmit, using the channel itself for coordination (no separate control channel).

Ideal MAC (rate R): (1) one active node gets full R; (2) M active nodes each get R/M on average; (3) fully decentralized; (4) simple.

Three classes:

1. Channel partitioning

- **TDMA (time division):** each node gets a fixed time slot per round; unused slots go idle. Efficient at high load, wasteful at low load.
- **FDMA (frequency division):** each node gets a fixed frequency band; unused bands idle.

2. Random access (allow collisions, then recover)

- **Slotted ALOHA:** synchronized slots; transmit a fresh frame in the next slot; on collision, retransmit in each later slot with probability p. **Max efficiency ≈ 1/e ≈ 37%.**
- **Pure (unslotted) ALOHA:** transmit immediately on arrival; a frame at t■ collides with anything sent in `[t■−1, t■+1]`. **Max efficiency ≈ 1/(2e) ≈ 18%.**
- **CSMA (carrier sense):** listen before transmit — if idle send, if busy defer. Collisions can still happen because of **propagation delay** (two nodes may not yet hear each other).
- **CSMA/CD (collision detection):** abort as soon as a collision is detected → less wasted time. Easy on wired links, hard on wireless. After the *m*-th collision, choose K randomly from `{0,1,…,2^m−1}` and wait **K·512 bit-times** (**binary exponential backoff**). Efficiency → 1 as `t_prop → 0` or `t_trans → ∞`.

3. "Taking turns" (best of both worlds)

- **Polling:** a central controller invites nodes to transmit in turn (used by Bluetooth). Concerns: polling overhead, latency, single point of failure.
- **Token passing:** a control token circulates; a node transmits only while holding it. Concerns: token overhead, latency, token loss.

> Ethernet uses **CSMA/CD**; 802.11 WiFi uses **CSMA/CA** (collision *avoidance*).

6.4 LANs: MAC addressing and ARP

MAC (physical / LAN) address: 48 bits, burned into the NIC ROM, administered by IEEE (manufacturers buy blocks). Used to move a frame between two interfaces on the **same subnet**.

- **IP address (32-bit) = "postal address"**: hierarchical, depends on the subnet; **not portable**.
- **MAC address = "Social Security number"**: flat; **portable** — move the NIC to another LAN and it keeps its MAC.

ARP — Address Resolution Protocol answers "given an IP address on my subnet, what is its MAC address?"

- Each node keeps an **ARP table**: `<IP, MAC, TTL>` (TTL typically ~20 min).
- If the target's MAC is unknown, the node **broadcasts** an ARP query (dest MAC `FF-FF-FF-FF-FF-FF`). The owner of that IP replies with a **unicast** ARP reply giving its MAC.

Sending to another subnet (A → B via router R): A frames the datagram (IP src = A, IP dest = B) with **destination MAC = R's MAC** (the first-hop router). R strips the frame, looks up the datagram, and re-frames it with **destination MAC = B's MAC**. The **IP addresses never change**; the **MAC addresses change hop by hop**.

6.5 Ethernet

The dominant wired LAN technology: cheap, simple, and it kept up with the speed race (10 Mbps → 400 Gbps). All speeds share a **common MAC protocol and frame format**.

Topology: old = **bus** (all nodes in one collision domain); today = **switched** star (each spoke is its own collision domain, no collisions).

Ethernet frame fields:

- **Preamble** (8 bytes): 7×`10101010` + `10101011`, to synchronize receiver/sender clocks.
- **Dest MAC** and **Source MAC** (6 bytes each). A NIC keeps a frame only if the dest MAC matches its own or is the broadcast address; otherwise it discards it.
- **Type:** identifies the upper-layer protocol (usually IP) — used to demux at the receiver.
- **Data / payload** (46–1500 bytes).
- **CRC:** error check; a frame failing CRC is dropped.

Ethernet is connectionless and unreliable: no handshake, no ACK/NAK. Lost data is recovered only if a higher layer (e.g., TCP) does it. Ethernet's MAC protocol is **unslotted CSMA/CD with binary backoff**.

6.6 Switches

A **switch** is a link-layer device that **stores and forwards** frames. It is **transparent** (hosts don't know it exists) and **plug-and-play / self-learning** (no configuration).

- Hosts have **dedicated** full-duplex links to the switch → **no collisions**; each link is its own collision domain. Multiple pairs (A→A′ and B→B′) can transmit **simultaneously**.

Self-learning switch table — entries `<MAC, interface, timestamp>`:

1. On a received frame, record `<source MAC, incoming interface>` (learn where the sender lives).
2. Look up the **destination MAC** in the table.
3. If found: if it's on the **same** interface the frame arrived on → **drop**; else **forward** on that one interface.
4. If not found: **flood** — forward on all interfaces except the arriving one.

Switches vs. routers: both are store-and-forward with forwarding tables.

- **Router:** network-layer (layer 3), examines IP headers, builds tables with **routing algorithms**.
- **Switch:** link-layer (layer 2), examines MAC headers, builds tables by **flooding + learning**.

6.7 VLANs and link virtualization

VLANs (Virtual LANs): configure one physical switch into multiple virtual LANs to fix scaling (all layer-2 broadcast traffic crossing the whole LAN), security, and administrative issues (a user physically moving but staying in the same logical LAN).

- **Port-based VLAN:** group switch ports (e.g., ports 1–8 = EE, 9–15 = CS); traffic is isolated between groups.
- **Trunk ports** carry frames of multiple VLANs between switches using **802.1Q**, which inserts a 4-byte tag (a 12-bit **VLAN ID**, so up to **$2^{12} = 4096$** VLANs, plus a 3-bit priority) and recomputes the CRC.

MPLS (Multiprotocol Label Switching): high-speed forwarding using a fixed-length **label** instead of longest-prefix IP lookup (VC-flavored, but the IP header is still present). Enables **traffic engineering** (route by source+dest, not just dest) and **fast reroute** on link failure.

6.8 Data center networking & a day in the life

Data centers: tens to hundreds of thousands of hosts. Structure: **server racks** → **Top-of-Rack (ToR / access) switches** → **tier-2 aggregation** → **tier-1 / spine** (leaf-spine, e.g., Facebook F16). Rich multipath gives more throughput and redundancy. **Load balancers** do application-layer routing of client requests.

A day in the life of a web request (student laptop → www.google.com), the grand synthesis:

1. **DHCP** (over UDP/IP/Ethernet, broadcast) → laptop gets its IP, first-hop router IP, DNS server IP.
2. **ARP** → laptop learns the first-hop router's MAC to send frames off-subnet.
3. **DNS** query (UDP) routed to DNS server → returns google's IP.
4. **TCP 3-way handshake** to google's web server.
5. **HTTP GET** over TCP → server returns the page → browser renders it.

PART C — SOLVED ASSIGNED QUESTIONS

Chapter 3 — Review Questions

R3. How is a UDP socket fully identified? A TCP socket? What is the difference?

- **UDP socket:** fully identified by a **2-tuple** — `(destination IP address, destination port number)`. Any two segments with the same destination IP and port are delivered to the **same** UDP socket, regardless of their source IP/port.
- **TCP socket:** fully identified by a **4-tuple** — `(source IP, source port, destination IP, destination port)`.
- **Difference:** UDP uses only the destination pair, so the source is ignored for demultiplexing. TCP uses all four values, so different clients (different source IP/port) hitting the same server port are separated into **different sockets**.

R4. Why might an application developer choose UDP over TCP?

Because UDP gives the application **finer control and lower overhead**:

- **No congestion control** — TCP throttles the sending rate during congestion; UDP keeps sending at the app's chosen rate (good for rate-sensitive, loss-tolerant media).
- **No connection setup** — TCP's 3-way handshake adds an RTT of delay before any data; UDP sends immediately.
- **No connection state** — a server can support many more active clients (no buffers, seq/ACK numbers, or window state per client).
- **Smaller header** (8 bytes vs 20) and the app can add exactly the reliability it needs (e.g., HTTP/3 over UDP).

R14. True or False?

	Statement	Answer	Why
a	B won't ACK A because it can't piggyback ACKs on data	**False**	TCP sends **pure ACK segments** with no data whenever needed.
b	rwnd never changes during the connection	**False**	rwnd changes as the receiver's buffer fills and drains.
c	Unacknowledged bytes A sends can't exceed the receive-buffer size	**True**	Flow control caps in-flight data at rwnd ≤ RcvBuffer.
d	If a segment's seq # is m, the next segment's seq # is necessarily m+1	**False**	The next seq # = m + (bytes of data in this segment), not m+1.
e	The TCP segment header has a field for rwnd	**True**	The 16-bit **receive window** field.

	Statement	Answer	Why
f	If the last SampleRTT = 1 s, then TimeoutInterval ≥ 1 s necessarily	**False**	Timeout uses **EstimatedRTT + 4·DevRTT** (a smoothed average), which can be below one noisy sample.
g	A sends seq=38 with 4 bytes; the ACK number is necessarily 42	**False**	42 would be the ACK **B sends back**. The ACK field *in A's own segment* is unrelated to its data (it acknowledges B's stream).

R15. Two back-to-back segments: first seq # = 90, second seq # = 110.

(a) How much data is in the first segment? The next segment's seq # is the byte-stream number right after the first segment's data, so:

```
data in first segment = 110 − 90 = 20 bytes
```

(b) First segment lost, second arrives at B. What ACK number does B send? TCP uses **cumulative ACKs** and acknowledges the next in-order byte expected. Bytes 90–109 are still missing, so B keeps asking for byte **90**:

```
acknowledgment number = 90
```

(This is a duplicate ACK — the mechanism that triggers fast retransmit.)

R18. True or false? When the timer expires (timeout) at the sender, ssthresh is set to one half of its previous value.

False. On a loss event, `ssthresh` is set to **one half of the current congestion window (cwnd)**, not half of the previous *ssthresh*. (And on a **timeout** specifically, cwnd is then reset to **1 MSS** and slow start restarts.)

Chapter 3 — Problems

P1. Client A and Client B each open a Telnet session to Server S. Give possible source/dest port numbers for the segments.

Telnet's well-known server port is **23**. Each client picks an arbitrary ephemeral source port; the two clients' source ports must differ (or, if same, the client IPs differ) so the server can separate the connections.

Segment direction	Source port	Destination port
a. A → S	467 (any ephemeral)	23
b. B → S	513 (any ephemeral, ≠ A's if same host)	23
c. S → A	23	467 (A's source port from a)
d. S → B	23	513 (B's source port from b)

e. If A and B are different hosts, can the source port numbers in (a) and (b) be the same? Yes. A and B have different IP addresses, so even with identical source ports the server's **4-tuples** (`srcIP`, `srcPort`, `dstIP`, `dstPort`) differ, and the two connections map to different sockets.

f. What if A and B are the same host? Then the source ports **must be different**, otherwise the two Telnet sessions from the same host would be indistinguishable (identical 4-tuples).

P27. Hosts A and B communicate over TCP; B has already received all bytes up through byte 126. A sends two back-to-back segments of 80 and 40 bytes. First segment: seq = 127, src port = 302, dst port = 80.

a. In the first segment: sequence number = **127**, source port = **302**, destination port = **80**.

b. Second segment (sent right after the first): its sequence number is the byte after the first segment's data:

```
seq = 127 + 80 = 207,   source port = 302,   destination port = 80,   40 bytes of data
```

c. If the first segment (seq 127) arrives before the second, B's ACK acknowledges the next expected byte after the 80 bytes:

```
ACK number = 127 + 80 = 207,   source port = 80,   destination port = 302
```

d. If the second segment (seq 207) arrives *before* the first, **there is a gap (bytes 127–206 missing). B sends a** cumulative ACK still asking for byte 127**:

```
ACK number = 127
```

e. Timeline when the first (seq 127) is lost, the second (seq 207) arrives, GBN-style / TCP cumulative ACK:

```
Host A                          Host B
---- seq=127, 80 bytes ---X     (lost)
---- seq=207, 40 bytes ------> arrives out of order
                         <----- ACK=127  (duplicate / gap ACK, still wants 127)
(timeout or 3 dup-ACKs)
---- seq=127, 80 bytes ------> gap filled
                         <----- ACK=247  (127+80+40, all data now received)
```

After the retransmitted segment fills the gap, the cumulative ACK jumps to **247** (= 127 + 80 + 40), covering both segments at once.

P28. Host A sends the same file to Host B. GBN with a window and sequence-number space — showing why cumulative ACKs matter.

The teaching point of P28 is TCP's **cumulative ACK behavior with lost ACKs and retransmissions**:

- TCP acknowledges the **next expected in-order byte**, so a single ACK can **cover several earlier segments** (and earlier lost ACKs).
- If ACKs are lost but a later ACK arrives with a **higher ACK number**, the sender still learns everything up to that number was received — no retransmission needed.
- The sender only retransmits when the **timer expires** or it sees **3 duplicate ACKs** (fast retransmit).

Worked mini-example (typical P28 setup): A sends 3 segments (seq 1, 2, 3 in segment-units); all arrive but ACK1 and ACK2 are lost while ACK3 (cumulative) arrives. Because ACK3 acknowledges everything up to and including segment 3, A advances its window past all three with the single ACK — the lost ACK1/ACK2 cause **no** retransmission. This shows the robustness of cumulative acknowledgment.

> If your printed P28 gives specific segment/ACK numbers, apply the same rule: **ACK number = 1 + (highest in-order byte received); a higher cumulative ACK retires all lower unACKed segments.**

P40. TCP congestion-window evolution (read from the cwnd-vs-round graph).

Standard version of this figure (rounds 1–26; ssthresh initially 32, later 21):

a. Intervals where TCP slow start is operating: `[1, 6]` and `[23, 26]` — cwnd grows **exponentially** (doubles each RTT) from 1 MSS up to ssthresh.

b. Intervals where congestion avoidance is operating: `[6, 16]` and `[17, 22]` — cwnd grows **linearly** (+1 MSS per RTT).

c. After the 16th transmission round, is loss detected by a triple duplicate ACK or a timeout? The window is **halved** (drops but stays > 1), and TCP continues in **congestion avoidance** → loss was detected by a **triple duplicate ACK** (TCP Reno behavior).

d. After the 22nd round, is loss detected by triple duplicate ACK or timeout? cwnd **drops to 1 MSS** and slow start restarts → loss detected by a **timeout**.

e. What is ssthresh initially (first round)? Slow start ends and congestion avoidance begins at round 6 with cwnd = 32, so **ssthresh = 32**.

f. What is ssthresh set to after the 18th round? Loss occurred around round 16 when cwnd = 42, so `ssthresh = 42/2 =` **21**; that value holds at round 18.

g. After the 24th round, which segment is sent? During rounds `[1,6]` cwnd = 1,2,4,8,16,32 → **63** segments sent through round 6. Continuing the additive increase through round 22, then slow start from round 23: sum the per-round cwnd values up to round 23, and the first segment sent in round 24 is the **next one after that running total**. (Plug in the exact cwnd values from your figure: `segments sent in rounds 1..23, then +1`.)

h. If loss is detected after round 26 by triple duplicate ACK, what are cwnd and ssthresh? Take the cwnd value at round 26 (call it W): after a triple-dup-ACK loss, **ssthresh = W/2** and **cwnd = W/2** (Reno halves and stays in congestion avoidance).

Method to memorize for P40-type questions: exponential (doubling) segment = **slow start**; linear (+1) segment = **congestion avoidance**; window **halved** on loss = **triple duplicate ACK**; window **dropped to 1** on loss = **timeout**; on any loss **ssthresh = cwnd/2**.

Chapter 6 — Review Questions

R1. In the transportation analogy (§6.1.1), if the passenger is a datagram, what is analogous to the link-layer frame?

The **transportation mode** — the car, bus, train, or plane that actually carries the passenger over one leg. (The passenger/datagram is *inside* the vehicle/frame for that hop, just as a datagram rides inside a frame across one link.)

R2. If every link provided reliable delivery, would TCP's reliable delivery be redundant?

No, TCP is still needed. Reliable *per-link* delivery only guarantees each datagram crosses **one link** without bit errors. It does **not** guarantee:

- **In-order end-to-end delivery** — datagrams of one TCP connection can take **different routes** and arrive out of order.
- **No loss** — datagrams can still be lost to **routing loops, buffer overflow at routers, or equipment failure** (events between links, not on a link).

TCP provides the ordered, loss-free **byte stream** to the application end-to-end, which link-layer reliability alone cannot.

R3. What services can a link-layer protocol offer the network layer? Which have corresponding services in IP? In TCP?

Link-layer service	Also in IP?	Also in TCP?
Framing	(framing/encapsulation exists in IP & TCP)	✓ (segment structure)
Link access	—	—
Reliable delivery	✗	✓
Flow control	✗	✓
Error detection	✓ (header checksum)	✓ (checksum)
Error correction	✗	✗
Full duplex	—	✓

Key point: **error detection** appears in IP and TCP; **reliable delivery, flow control, and full duplex** correspond to TCP; **error correction and link access** are essentially link-layer-only.

R4. Two nodes start transmitting a length-L packet at the same time over a broadcast channel of rate R, with propagation delay d_prop. Is there a collision if d_prop < L/R?

Yes, there is a collision. L/R is the transmission time of the packet. If `d_prop < L/R`, then a node is **still transmitting** its own frame when the other node's frame arrives (the signal reaches it before it finishes sending). Two signals overlap on the medium at the same time → **collision**.

R10. Nodes A, B, C share a broadcast LAN. A sends thousands of datagrams to B, each frame addressed to B's MAC. Will C's adapter process these frames? Pass them up? What changes if A uses the broadcast MAC address?

- **Unicast to B's MAC:** C's adapter **receives and processes** each frame (checks the destination MAC), sees it does **not** match its own address, and **discards** it — the datagrams are **not** passed up to C's network layer.
- **Broadcast MAC (`FF-FF-FF-FF-FF-FF`):** C's adapter **both processes the frames and passes the datagrams up** to its network layer, because a broadcast address matches every adapter.

R11. Why is an ARP query sent in a broadcast frame, but an ARP reply in a frame with a specific destination MAC?

- **Query is broadcast** because the querying host **doesn't yet know** which adapter (MAC) owns the target IP — it must ask **everyone** on the LAN, so it uses the broadcast MAC so all adapters process it.
- **Reply is unicast** because the responder **already knows** the querier's MAC (it was in the query), so it sends the reply **directly** to that one node — no need to bother every other node on the LAN with a broadcast.

QUICK-REFERENCE FORMULA SHEET

```
Transmission delay      d_trans = L / R
Propagation delay       d_prop  = d / s
Stop-and-wait util.     U = (L/R) / (RTT + L/R)
RTT estimate            EstimatedRTT = (1-α)EstRTT + α·SampleRTT     (α=0.125)
RTT deviation           DevRTT = (1-β)DevRTT + β|SampleRTT-EstRTT|    (β=0.25)
Timeout                 TimeoutInterval = EstimatedRTT + 4·DevRTT
TCP throughput (avg)    ≈ (3/4)·W / RTT
SR window rule          window ≤ (seq-number space)/2
Slotted ALOHA max eff.  1/e  ≈ 0.37
Pure ALOHA max eff.     1/(2e) ≈ 0.18
CSMA/CD backoff         K ∈ {0,…,2^m−1} after m-th collision; wait K·512 bit-times
CRC                     choose R so <D,R> divisible by G (mod 2); detects bursts ≤ r
802.1Q VLANs            max = 2^12 = 4096
MAC / IPv4 / IPv6       2^48 / 2^32 / 2^128 addresses
```

Top exam traps (memorize)

1. **Flow control** protects the receiver (rwnd); **congestion control** protects the network (cwnd). Different things.
2. **Next TCP seq # = current seq # + bytes of data**, not +1.
3. On loss, **ssthresh = cwnd / 2**. Timeout → cwnd = 1 MSS (slow start). Triple-dup-ACK → cwnd = cwnd/2 (Reno).
4. **UDP demux = 2-tuple; TCP demux = 4-tuple.**
5. **IP addresses stay fixed end-to-end; MAC addresses change every hop.**
6. ARP **query = broadcast**, ARP **reply = unicast**.
7. Switch on unknown dest → **flood**; on known dest same-port → **drop**; else forward on one port.
8. Slow start = exponential; congestion avoidance = linear.
9. **d_trans = L/R** depends on L and R; **d_prop = d/s** depends on distance, NOT on L or R. Only **d_queue** is variable.
10. **Throughput = min of link rates** (the bottleneck). File time = F / throughput.
11. **Usable hosts = 2■ – 2** (reserve network + broadcast). **Block size = 256 – last mask octet.**
12. VLSM: allocate **largest subnet first**. Non-persistent HTTP ≈ **2·RTT per object**; persistent reuses the connection.